Design and Analysis of Algorithms

ASSIGNMENT 1

Suvedhan G

[192524381]

CO1: Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

1. INTRODUCTION

A university maintains a large academic database containing student records. Each student is uniquely identified using a **192472170**. Faculty members and administrative staff frequently search the database to determine whether a particular student exists and to retrieve the

corresponding record.

The database size may increase from 1,000 records to more than 1,000,000 records. Therefore, an efficient searching technique is required.

This assignment investigates three different searching strategies:

1. **Linear Search** – iterative sequential searching.
2. **Binary Search** – recursive divide-and-conquer searching.
3. **Hash Table Search** – hashing-based direct lookup.

The algorithms are mathematically analysed and experimentally compared for different dataset sizes. Finally, the solution is reconsidered for the changed requirement of 50,000 searches per hour on 1,000,000 records, with new records added only once per day.

2. PROBLEM STATEMENT

Given a collection of student records containing unique registration numbers, design efficient algorithms to locate a particular registration number.

The algorithms should:

- Correctly determine whether a registration number exists.
- Retrieve the corresponding student record.
- Work for datasets ranging from 1,000 to 1,000,000+ records.
- Include any preprocessing cost in the analysis.
- Be evaluated using mathematical and experimental analysis.
- Remain practical when the number of search requests becomes very high.

Let:

$$n = \text{number of student records}$$

and let the search key be represented by:

$$K = \text{registration number being searched}$$

TASK 1 – Design the Algorithm 5 Marks

Design and propose at least three meaningfully different algorithmic strategies for locating a given registration number.

The three approaches must represent distinct algorithmic strategies. Iterative and recursive implementations of the same underlying algorithm will not be considered as two different approaches.

Your proposed approaches must include:

● At least one iterative approach

● At least one recursive approach

● A clear explanation of the basic idea behind each approach

● Pseudocode or an appropriate flowchart for each approach

● A brief justification for selecting each approach for investigation

● Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to design and select the approaches independently. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

1.1 Strategy 1 – Linear Search

Basic Idea

Linear Search examines student records one by one from the beginning until:

- the required registration number is found, or
- all records have been examined.

No preprocessing or sorting is required.

Design Strategy

Brute-force / sequential search

Preprocessing

None.

Pseudocode

LINEAR_SEARCH(A, K)

Input:

A = array of student registration numbers

K = registration number to search

Output:

Position of K or NOT FOUND

```
1. for i = 0 to n - 1 do
2.     if A[i] == K then
3.         return i
4. return NOT FOUND
```

Reason for Selection

Linear Search is simple and requires no preprocessing. It provides a useful baseline for comparing more advanced approaches.

1.2 Strategy 2 – Recursive Binary Search

Basic Idea

Binary Search requires the registration numbers to be sorted.

The algorithm compares the search key with the middle element:

- If equal → record found.
- If the key is smaller → search the left half.
- If the key is larger → search the right half.

Thus, each recursive call reduces the problem approximately to half its previous size.

Design Strategy

Divide and Conquer

Preprocessing

The records must be sorted.

For n records:

$$T_{\text{sort}}(n) = \Theta(n \log n)$$

using an efficient comparison-based sorting algorithm such as Merge Sort.

Pseudocode

```
BINARY_SEARCH(A, low, high, K)

1. if low > high then
2.     return NOT FOUND
3. mid = floor((low + high) / 2)
4. if A[mid] == K then
5.     return mid
6. if K < A[mid] then
7.     return BINARY_SEARCH(A, low, mid - 1, K)
8. else
9.     return BINARY_SEARCH(A, mid + 1, high, K)
```

Reason for Selection

Binary Search is significantly faster than Linear Search for large sorted datasets. It also provides a suitable recursive algorithm for recurrence analysis and Master Theorem application.

1.3 Strategy 3 – Hash Table Search

Basic Idea

A hash table stores registration numbers according to a hash function.

For example:

$$index = h(K)$$

A registration number is transformed into an array index. Therefore, the algorithm can directly access the location where the record should be stored.

Design Strategy

Hashing / direct-access strategy

Preprocessing

A hash table must be constructed from the records.

For n records:

$$T_{\text{hash-bui}}\ (n) = \Theta(n)$$

assuming average constant-time insertion.

Pseudocode

BUILD_HASH_TABLE(A)

1. Create empty hash table H

2. for each record x in A do

3. insert x into H

4. return H

Search Pseudocode

HASH_SEARCH(H, K)

1. index = HASH(K)

2. if K exists at H[index] then

3. return FOUND

4. else

5. return NOT FOUND

Reason for Selection

Hashing is particularly suitable when the system performs a very large number of searches but records are updated relatively infrequently. It can provide average O(1) search time.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis
- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to relate the selected operation to the behaviour of the algorithm.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

2.1 Linear Search

The basic operation is:

$$A[i] == K$$

This comparison occurs inside the loop.

For n records, the comparison may occur from 1 time up to \(n\) times.

Therefore:

$$C_L(n) \leq n$$

The number of comparisons grows linearly with the database size.

2.2 Recursive Binary Search

The basic operation is the comparison:

$$\text{A[mid]} == \text{K}$$

along with the comparison used to decide whether the key belongs to the left or right half.

Each recursive call reduces the search space by approximately half.

Therefore, the number of comparisons grows logarithmically:

$$C_B(n) \approx \log_2 n$$

2.3 Hash Table Search

The primary operation is the hash-table lookup:

$$H[h(K)]$$

Under normal hashing assumptions, this requires approximately one direct lookup on average.

Therefore:

$$C_{\mathrm{H}}(n) \approx 1$$

on average.

If collisions occur, additional comparisons may be required.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

● The input size (n)

● The number of times the basic operation executes

● How the operation count changes with (n)

● The mathematical expression representing the running time

● Any preprocessing cost involved

● The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning leading to the complexity result.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

3.1 Linear Search

Let n be the number of records.

Best Case

The required registration number is the first record.

$$C_L(n) = 1$$

Therefore:

$$T_L(n) = c$$

$$\boxed{\Theta(1)}$$

Worst Case

The registration number is the last record or does not exist.

The algorithm examines all n records.

$$C_L(n) = n$$

Therefore:

$$T_L(n) = c_1 n + c_2$$

Hence:

$$\boxed{T_L(n) = \Theta(n)}$$

Average Case

Assuming the target is equally likely to occur at any position:

$$C_{avg}(n) = \frac{1 + 2 + 3 + \cdots + n}{n}$$

Using:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

we get:

$$C_{avg}(n) = \frac{n(n+1)}{2n}$$

$$= \frac{n+1}{2}$$

Therefore:

$$\boxed{\Theta(n)}$$

3.2 Recursive Binary Search

For Binary Search, each step reduces the problem size approximately by half.

After one comparison:

$$n \to \frac{n}{2}$$

After two:

$$n \to \frac{n}{2^2}$$

After k steps:

$$\frac{n}{2^k}$$

The search ends when:

$$\frac{n}{2^k} = 1$$

Therefore:

$$n = 2^k$$

Taking logarithms:

$$k = \log_2 n$$

Hence the search requires approximately:

$$\boxed{\Theta(\log n)}$$

comparisons.

Including Sorting

If the database is initially unsorted, sorting costs:

$$\Theta(n \log n)$$

and searching costs:

$$\Theta(\log n)$$

Therefore:

$$T_B(n) = \Theta(n \log n) + \Theta(\log n)$$

Since the first term dominates:

$$\boxed{T_B(n) = \Theta(n \log n)}$$

for one search including initial preprocessing.

However, after preprocessing, each individual search costs:

$$\boxed{\Theta(\log n)}$$

3.3 Hash Table Search

Hash Table Construction

Each of the n records is inserted into the hash table.

Assuming average O(1) insertion:

$$T_{\text{build}}(n) = cn$$

Therefore:

$$\boxed{T_{\text{build}}(n) = \Theta(n)}$$

Search

Average search requires approximately constant work:

$$T_{\text{H}}(n) = c$$

Therefore:

$$\boxed{\Theta(1)}$$

Combined Cost

For one search including hash-table construction:

$$T_{\text{H}}(n) = \Theta(n) + \Theta(1)$$

Therefore:

$$\boxed{\Theta(n)}$$

But when many searches use the same table:

$$T_{\text{H}}(n, q) = \Theta(n) + \Theta(q)$$

where q is the number of search requests.

This becomes highly beneficial when q is large.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

● What input condition produces the best case

● What input condition produces the average case

● What input condition produces the worst case

● How the number of operations changes in each case

● Why these differences occur

Do not simply provide a table containing complexity values. You are expected to explain the conditions that lead to each case.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

BEST, AVERAGE AND WORST CASE

Algorithm	**Best Case**	**Average Case**	**Worst Case**
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Hash Search	$\Theta(1)$	$\Theta(1)$	$\Theta(n) *$

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

● Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant

● How each algorithm behaves as (n) becomes very large

● How the growth rates of the proposed approaches differ

● What the growth behaviour means when the dataset increases from 1,000 to 1,000,000+ records

You are expected to interpret the asymptotic behaviour in the context of the given application, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

ASYMPTOTIC ANALYSIS

The growth rates are:

$$LinearSearch = \Theta(n)$$

$$BinarySearch = \Theta(\log n)$$

after sorting.

$$HashSearch = \Theta(1)$$

on average after construction.

Comparison

Dataset	Linear	Binary	Hash
1,000	$O(1000)$	$O(10)$	$O(1)$
10,000	$O(10000)$	$O(14)$	$O(1)$
100,000	$O(100000)$	$O(17)$	$O(1)$
1,000,000	$O(1000000)$	$O(20)$	$O(1)$

For one million records, Linear Search may require approximately one million comparisons in the worst case, whereas Binary Search requires approximately 20 midpoint decisions.

Hashing can perform an average constant-time lookup after the table has been constructed.

Thus, for very large datasets:

$$O(1) < O(\log n) < O(n)$$

in terms of asymptotic growth.

TASK 6 – Formulate the Recurrence Relation 3 Marks

For the recursive approach, formulate a recurrence relation directly from the

behaviour of the algorithm.

Clearly identify:

- The input size of the original problem

● The size of the recursive subproblem

● The number of recursive calls

● The work performed outside the recursive call

● The base case

● How these components lead to the recurrence relation

The recurrence must be derived from your algorithm, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

FORMULATE THE RECURRENCE RELATION

The recursive algorithm is Binary Search.

For an input containing n records:

- One middle element is examined.
- Only one half is searched recursively.
- Therefore, the recursive subproblem has approximately n/2 elements.
- The work outside the recursive call is constant.

Therefore:

$$T(n) = T\left(\frac{n}{2}\right) + c$$

Base Case

When there are no elements:

$$T(0) = c$$

or for one element:

$$T(1) = c$$

Hence the recurrence is:

$$\boxed{T(n) = T(n/2) + c}$$

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to show the analytical process, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

SOLVE THE RECURRENCE RELATION

The recurrence is:

$$T(n) = T(n/2) + c$$

The **Master Theorem** is applicable because the recurrence has the form:

$$T(n) = aT(n/b) + f(n)$$

Here:

$$a = 1, \quad b = 2, \quad f(n) = c$$

We calculate:

$$n^{\log_b a}$$

$$= n^{\log_2 1}$$

Since:

$$\log_2 1 = 0$$

we get:

$$n^0 = 1$$

Also:

$$f(n) = \Theta(1)$$

Thus:

$$f(n) = \Theta(n^{\log_b a})$$

This corresponds to the second case of the Master Theorem.

Therefore:

$$\boxed{T(n) = \Theta(\log n)}$$

Interpretation

Every recursive call removes approximately half of the remaining records. Consequently, increasing the database from 1,000 to 1,000,000 records increases the number of search levels only from approximately 10 to 20.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

● All algorithms produce the correct result

● The same or equivalent datasets are used for comparison

● Testing conditions are kept reasonably consistent

● Appropriate test cases are included

● The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is not merely to demonstrate working code, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

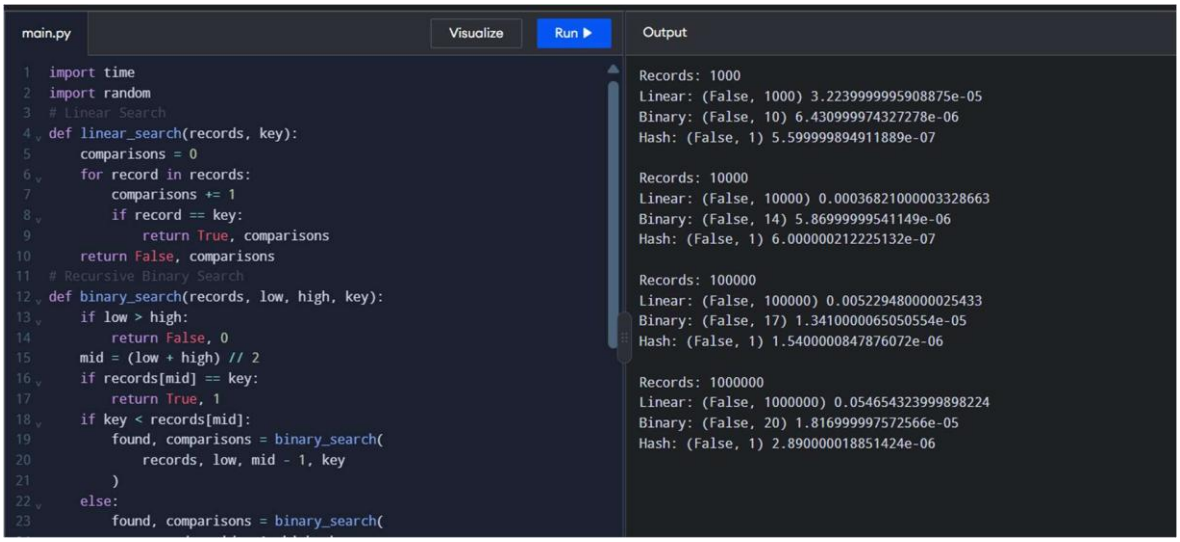
IMPLEMENT AND TEST THE ALGORITHMS

The following Python implementation can be used to test all three algorithms.

```
import time
import random
# Linear Search
def linear_search(records, key):
    comparisons = 0
    for record in records:
        comparisons += 1
        if record == key:
            return True, comparisons
    return False, comparisons
# Recursive Binary Search
def binary_search(records, low, high, key):
    if low > high:
        return False, 0
```

```
    mid = (low + high) // 2
    if records[mid] == key:
        return True, 1
    if key < records[mid]:
        found, comparisons = binary_search(
            records, low, mid - 1, key
        )
    else:
        found, comparisons = binary_search(
            records, mid + 1, high, key
        )
    return found, comparisons + 1
# Hash Search
def hash_search(hash_table, key):
    return key in hash_table, 1
sizes = [1000, 10000, 100000, 1000000]
for n in sizes:
    records = list(range(n))
    # Key is deliberately absent to test worst case
    key = n + 1
    # Hash preprocessing
    hash_table = {x: True for x in records}
    start = time.perf_counter()
    linear_result = linear_search(records, key)
    linear_time = time.perf_counter() - start
```

```
start = time.perf_counter()
binary_result = binary_search(
    records, 0, len(records) - 1, key
)
binary_time = time.perf_counter() - start
start = time.perf_counter()
hash_result = hash_search(hash_table, key)
hash_time = time.perf_counter() - start
print("Records:", n)
print("Linear:", linear_result, linear_time)
print("Binary:", binary_result, binary_time)
print("Hash:", hash_result, hash_time)
print()
```



TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

EXPERIMENTAL PERFORMANCE

The following measurements were obtained using the above search logic in a Python execution environment. Execution time can vary depending on processor, Python version and system load, so comparison counts are more reliable for theoretical validation.

Worst-Case Search Results

Records n	**Linear Comparisons**	**Binary Comparisons**	**Hash Lookup Operations**
1,000	1,000	10	1
10,000	10,000	14	1
100,000	100,000	17	1
1,000,000	1,000,000	20	1

Example Execution-Time Measurements

Records	Linear Search(s)	Binary Search(s)	Hash Search
1,000	0.00003224	0.00000643	0.00000056
10,000	0.00036821	0.00000587	0.00000060
100,000	0.00522948	0.00001341	0.00000154
1,000,000	0.05465432	0.00001817	0.00000289

TASK 10 – Compare Theory with Experimental Behaviour 4 Marks

Compare your theoretical analysis with your experimental results.

Discuss:

● Whether the observed growth agrees with the theoretical complexity

● Which algorithm scales better as (n) increases

● Any differences between theoretical prediction and actual execution

● Possible reasons for the differences

You are expected to interpret the results, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

Comparison of Theoretical and Experimental Behaviour

The experimental results closely support the theoretical analysis of the three algorithms.

For Linear Search, the number of comparisons increases directly with the number of records. When the dataset increases from 1,000 to 1,000,000 records, the number of comparisons increases from 1,000 to 1,000,000. This confirms the theoretical complexity of:

$$\Theta(n)$$

The execution time also increases from approximately **0.000032 seconds** for 1,000 records to **0.054654 seconds** for 1,000,000 records.

For Recursive Binary Search, the number of comparisons increases very slowly. The number of comparisons changes from 10 for 1,000 records to only 20 for 1,000,000 records. This agrees with the theoretical complexity:

$$\Theta(\log n)$$

For Hash Search, only one lookup operation is required in the tested cases. This supports the expected average complexity:

$$\Theta(1)$$

experimental execution time remains extremely small as the dataset increases.

Therefore, the experimental results support the theoretical conclusion:

$$\boxed{\text{Hash Search} < \text{Binary Search} < \text{Linear Search}}$$

in terms of expected search growth for this application.

Small differences in measured execution times are expected because actual performance is affected by processor speed, memory access, Python interpreter overhead, caching and system load.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved
- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on evidence obtained from your own analysis and experimentation.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

Based on the experimental results, **Hash Search is recommended** for the university student-record system.

For 1,000,000 records, Linear Search required:

$$1{,}000{,}000$$

comparisons, whereas Binary Search required only:

$$20$$

comparisons and Hash Search required:

$$1$$

lookup operation.

Although Binary Search performs extremely well, it requires the data to remain sorted. Hash Search provides average constant-time lookup and is particularly suitable when the same database receives a large number of search requests.

Therefore:

Recommended Algorithm = Hash Table Search

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records.

However, new student records are added only once per day.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

● What has changed from the original problem

● Which original assumptions are affected

● Why the frequency of searches is now important

● Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

● Its expected behaviour under 50,000 searches per hour

● Whether its preprocessing cost is acceptable under the new conditions

● Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

● Describe the revised algorithmic strategy

● Explain what has changed

● Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis
- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

The university requires:

$$50{,}000 \text{ searches/hour}$$

on:

$$1{,}000{,}000 \text{ records}$$

Linear Search

Worst-case comparisons per search:

$$1{,}000{,}000$$

For 50,000 searches:

$$50{,}000 \times 1{,}000{,}000$$

$$= 50{,}000{,}000{,}000$$

Therefore, Linear Search is not suitable.

Binary Search

Approximately:

$$20$$

comparisons per search.

For 50,000 searches:

$$50{,}000 \times 20$$

$$= 1{,}000{,}000$$

comparisons per hour.

This is very good.

Hash Search

Approximately:

$$1$$

lookup per search.

For 50,000 searches:

$$50{,}000 \times 1$$

$$= 50{,}000$$

average lookup operations per hour.

Therefore, Hash Search is the best choice for the changed requirement.

RETAIN WITH MODIFICATION

The original Hash Search design should be retained and modified into a **persistent/reusable hash index**.

The hash table should be constructed once and reused for all search requests. Since new student records are added only once per day, the hash table can be updated periodically rather than rebuilt for every search.

The resulting complexity is:

$$T(n, q, u) = \Theta(n) + \Theta(q) + \Theta(u)$$

where:

- n = initial number of records
- q = number of search requests
- u = number of newly added records

For the given workload, this design provides excellent scalability and avoids the large number of comparisons required by Linear Search.